If it is triggered, the session garbage collection will check the last access time of all sessions, and if they exceed the value of `session.gc_maxlifetime` (default value is `1440` seconds to 24 minutes), then those sessions are purged (the files are deleted, or the Redis keys are removed, and so on).

For large and/or very high traffic sites you'll probably want to either change `gc_probability` and `gc_divisor` or move the session clean-up away from requests entirely and use your operating system's scheduler to do the work. This could take the form of a simple bash script, called via `cron` on a Linux-based system, that checks file timestamps (or the memcache/Redis equivalent), or even better, you could run a PHP CLI script that uses `session_start()` in conjunction with `session_gc()`. If doing this, make sure you configure `session.gc_probability` to `0` (so it will never trigger during a user request).

You can test this in your cli-server container; just run `php -a` to run PHP in interactive shell mode:

```
root@cacfcef67c60:/usr/src/myapp# ls /tmp
sess_54a9a1f51c7842c0a58d0b971cb463d1  sess_963229a0f8fe5dd892a14ecf64
5b4c43
sess_71315a640a6f21e2f55773b24cef24bf  sess_
b91b04c21ca8e270aaab9e9b1d0db7c8

root@0d72be74121b:/usr/src/myapp# php -a
php > session_start();
php > session_gc();
php > ^C

root@cacfcef67c60:/usr/src/myapp# ls /tmp
sess_20fb90fe176daa80df6614c839c2f213
```

In the example above, all of the files in `/tmp` were over 24 minutes old. After running `session_start()`/`session_gc()`, only the new session file (created by the session started interactively) remains.

Functions

Table 11-6 presents a partial list of commonly used PHP session functions with a brief description of functionality.